



Universidad Nacional Autónoma de México

Facultad de Ingeniería

Materia: Organización Y Arquitectura de Computadoras

Profesor: M.I. José Antonio de Jesús Arredondo Garza

Práctica 2

Sistema de control de segundo orden

No. de Cuenta	**Nombre**
320068234	Ayala Hernández María Fernanda
320335163	Salazar Islas Luis Daniel
320078282	Tepal Briseño Hansel Yael
320223822	Ugartechea González Luis Antonio

Fecha de entrega: 12 de Octubre del 2026

Calificación: _____

1 Objetivo

En esta práctica se implementó en una Raspberry Pi Pico 2 W un sistema de control de segundo orden en tiempo discreto. El sistema lee la posición de un potenciómetro mediante el convertidor analógico-digital (ADC), calcula la salida usando una ecuación de diferencias y ejecuta el proceso cada $T_s = 0.025$ s, equivalente a una frecuencia de muestreo de $f_s = 40$ Hz.

El objetivo es relacionar la función de transferencia continua con una implementación digital en MicroPython y observar el comportamiento del sistema directamente sobre el hardware.

2 Fundamentos teóricos

Una función de transferencia de segundo orden puede escribirse como

$$H(s) = \frac{\omega_0}{s^2 + 2\omega_0 s + \omega_0^2}. \tag{1}$$

Para trabajar con la Raspberry Pi Pico 2 W, el sistema continuo debe discretizarse para obtener una función $H(z)$ y una ecuación de diferencias.

El periodo de muestreo utilizado es

$$T_s = 0.025 \text{ s}, \qquad f_s = \frac{1}{T_s} = 40 \text{ Hz}. \tag{2}$$

La implementación utiliza la forma discreta

$$H(z) = \frac{b_0 + b_1 z^{-1} + b_2 z^{-2}}{1 + a_1 z^{-1} + a_2 z^{-2}}. \tag{3}$$

Despejando la salida se obtiene la ecuación de diferencias

$$y[k] = b_0 x[k] + b_1 x[k-1] + b_2 x[k-2] - a_1 y[k-1] - a_2 y[k-2]. \tag{4}$$

En el programa se emplean los coeficientes

$$b_0 = 0.01, \quad b_1 = 0.02, \quad b_2 = 0.01, \qquad a_1 = -1.6, \quad a_2 = 0.64. \tag{5}$$

Por tanto, el denominador queda como

$$z^2 - 1.6z + 0.64 = (z - 0.8)^2. \tag{6}$$

Los polos tienen magnitud 0.8, por lo que se encuentran dentro del círculo unitario y el filtro discreto es estable. Además, la suma de los coeficientes del numerador es igual a la evaluación del denominador en $z = 1$, por lo que la ganancia de corriente directa es aproximadamente uno.

3 Desarrollo

El programa se divide en cuatro tareas principales:

- configurar el ADC del pin 26 para leer el potenciómetro;
- suavizar la lectura mediante un filtro exponencial;
- calcular la salida del sistema con la ecuación de diferencias;
- ejecutar las tareas periódicamente y mostrar la muestra actual.

El ADC de la Pico entrega valores entre 0 y 65535. Para reducir las variaciones rápidas del potenciómetro se utiliza un filtro de promedio exponencial:

$$x_f[k] = \alpha x_{ADC}[k] + (1-\alpha)x_f[k-1], \qquad \alpha = 0.1. \tag{7}$$

Un valor pequeño de α produce una lectura más suave, aunque también incrementa el tiempo de respuesta ante cambios del potenciómetro.

3.1 Estados del sistema

Las variables `x_k1` y `x_k2` almacenan las dos entradas anteriores, mientras que `y_k1` y `y_k2` almacenan las dos salidas anteriores. Al terminar cada iteración, los estados se desplazan para que estén disponibles en el siguiente periodo de muestreo.

3.2 Temporizador e interrupción

El temporizador se configura con un periodo de 25 ms:

```
1 timer_muestreo.init(period=25,
2                     mode=Timer.PERIODIC,
3                     callback=iteracion_control)
```

Cada interrupción ejecuta `iteracion_control`, lee el ADC, actualiza el filtro y calcula la salida. La interrupción no imprime directamente en la terminal: solo guarda la última muestra. El bucle principal se encarga de imprimirla, evitando operaciones de salida dentro de la interrupción.

3.3 Protecciones de estabilidad

El programa verifica que la salida sea finita mediante `isfinite`. Si se detecta `nan` o infinito, se reinician los estados de salida. También se limita la salida al intervalo del ADC, $[-65535, 65535]$, para evitar que una condición anormal se propague al resto del sistema.

4 Implementación en MicroPython

El código completo utilizado en la Raspberry Pi Pico 2 W es el siguiente.

```
'''
SISTEMA DE CONTROL DE SEGUNDO ORDEN

Esta implementación ejecuta una función de transferencia discreta H(z)
basada en un sistema de segundo orden continuo.
Se utiliza un temporizador por hardware de la Raspberry Pi Pico 2W
para garantizar un tiempo de muestreo estricto de Ts = 0.025s (40 Hz).

Por:
- Ayala Hernández María Fernanda
- Salazar Islas Luis Daniel
- Tepal Briseño Hansel Yael
- Ugartechea González Luis Antonio
'''

# --- EXPLICACIÓN DEL FUNCIONAMIENTO ---
# El script utiliza el módulo `machine.Timer` de MicroPython para
# generar una interrupción de hardware exacta cada 25 milisegundos (40 Hz),
# cumpliendo con la restricción de muestreo sin bloquear el microcontrolador.
# En cada interrupción (ciclo de reloj) el sistema realiza lo siguiente:
# 1. Lee la entrada actual x[k] (ej. desde un pin analógico).
# 2. Calcula la salida actual y[k] resolviendo la ecuación de diferencias obtenida al discretizar H(s) a H(z).
# 3. Actualiza el historial de variables (memoria) desplazando los valores anteriores (x[k-1], y[k-1], etc.) para que estén listos en el próximo ciclo.
# --------------- ARM Assembly function ---------------

from machine import Pin, Timer, ADC
from math import isfinite
import time

# --- COEFICIENTES DEL SISTEMA DISCRETO H(z) ---
# b0 = 0.01
# b1 = 0.02
# b2 = 0.01
# a1 = -1.6
# a2 = 0.64

# Sistema con respuesta más rápida y amortiguada:
```

```
# b0 = 0.04
# b1 = 0.08
# b2 = 0.04
# a1 = -1.2
# a2 = 0.36

# Sistema con respuesta más lenta:
b0 = 0.005
b1 = 0.010
b2 = 0.005
a1 = -1.8
a2 = 0.81

# --- VARIABLES DE ESTADO (Memoria del sistema) ---
x_k1 = 0.0
x_k2 = 0.0
y_k1 = 0.0
y_k2 = 0.0
entrada_filtrada = 0.0
ultima_entrada = 0
ultima_salida = 0.0
nueva_muestra = False

ALFA_ENTRADA = 0.1
LIMITE_SALIDA = 65535.0

entrada_adc = ADC(26)

def iteracion_control(timer):
    global x_k1, x_k2, y_k1, y_k2, entrada_filtrada
    global ultima_entrada, ultima_salida, nueva_muestra
    # 1. Leer la entrada actual x[k]
    # El ADC tiene un valor de 0 a 65535
    lectura_adc = entrada_adc.read_u16()
    entrada_filtrada = (ALFA_ENTRADA * lectura_adc) + ((1 -
       ALFA_ENTRADA) * entrada_filtrada)
    x_k = entrada_filtrada

    # 2. Calcular la salida y[k] mediante la ecuación de diferencias
    y_k = (b0 * x_k) + (b1 * x_k1) + (b2 * x_k2) - (a1 * y_k1) - (a2
        * y_k2)

    if not isfinite(y_k):
        y_k = 0.0
        y_k1 = 0.0
        y_k2 = 0.0
    elif y_k > LIMITE_SALIDA:
        y_k = LIMITE_SALIDA
```

```
84      elif y_k < -LIMITE_SALIDA:
85          y_k = -LIMITE_SALIDA
86
87      # Guardar la muestra para imprimirla fuera de la interrupción.
88      ultima_entrada = x_k
89      ultima_salida = y_k
90      nueva_muestra = True
91
92      # 4. Actualizar las variables de estado para la siguiente
            interrupción
93      x_k2 = x_k1
94      x_k1 = x_k
95      y_k2 = y_k1
96      y_k1 = y_k
97
98  # --- INICIALIZACIÓN DEL TIMER ---
99  tiempo_muestreo = 25 # En milisegundos
100 timer_muestreo = Timer(-1)
101 timer_muestreo.init(period=tiempo_muestreo, mode=Timer.PERIODIC,
        callback=iteracion_control)
102
103 frecuencia_muestreo = 1000 / tiempo_muestreo
104 print("Sistema de control de segundo orden iniciado a {:.2f} Hz.".
        format(frecuencia_muestreo))
105
106 # El bucle principal queda libre para otras tareas (pantallas,
        comunicaciones, etc.)
107 try:
108     while True:
109         if nueva_muestra:
110             nueva_muestra = False
111             # Formato consumido por la interfaz gráfica: DATA,
                    tiempo_ms,x,y.
112             print("DATA,{},{:.3f},{:.3f}".format(
113                 time.ticks_ms(), ultima_entrada, ultima_salida))
114         time.sleep_ms(1)
115 except KeyboardInterrupt:
116     # Detener el timer si se cancela la ejecución en la consola
117     timer_muestreo.deinit()
118     print()
119     print("Sistema detenido.")
```

Listing 1: Implementación del sistema de control

5 Conexión del potenciómetro

El potenciómetro se conecta entre 3.3 V y GND, y su terminal central se lleva al pin ADC26. Para disminuir el ruido eléctrico se recomienda colocar un capacitor de 100 nF entre ADC26 y GND, lo más cerca posible de la placa. Una resistencia serie de aproximadamente 1 kΩ puede colocarse entre el cursor del potenciómetro y el ADC para formar un filtro RC junto con el capacitor.

La Pico y el potenciómetro deben compartir la misma referencia de tierra. No debe aplicarse 5 V al ADC, ya que sus entradas trabajan con el nivel de 3.3 V.

6 Ejecución en la Raspberry Pi Pico

El programa se carga y ejecuta desde la carpeta `practica_02` con:

```
../upload.sh source/practica_02.py
```

El script de carga verifica que el archivo exista, libera el puerto serial si está ocupado, copia el programa a la Pico como `main.py` y lo ejecuta mediante `mpremote`.

```
#!/usr/bin/env bash

set -euo pipefail

script="${1:?Usage: ./upload.sh SCRIPT.py [DEVICE]}"
device="${2:-/dev/ttyACM0}"

if [[ ! -f "$script" ]]; then
  echo "File not found: $script" >&2
  exit 1
fi

if fuser "$device" >/dev/null 2>&1; then
  echo "Port $device is busy. Stopping the process using it..." >&2
  fuser -k "$device" >/dev/null 2>&1 || true
fi

mpremote connect "$device" fs cp "$script" :main.py
mpremote connect "$device" run "$script"
```

Listing 2: Script utilizado para cargar el programa

7 Interfaz de monitoreo y graficación

Para observar el comportamiento del sistema se desarrolló una interfaz en Python que recibe las muestras de la Raspberry Pi Pico 2 W mediante el puerto serial USB. El programa de la Pico envía una línea por muestra con el formato `DATA,tiempo_ms,entrada,salida`. La interfaz separa estos cuatro campos, convierte la entrada y la salida a valores numéricos y los dibuja en dos gráficas actualizadas en tiempo real.

La interfaz se ejecuta en la computadora, no en la Pico. Por esta razón, el puerto serial debe quedar libre después de cargar el programa. El monitor se inicia desde la carpeta `practica_02` con:

```
python3 graficar_practica_02.py /dev/ttyACM0
```

La ventana permite conectar y desconectar el puerto, pausar la visualización, limpiar el historial y guardar las muestras recibidas en un archivo CSV. El programa completo de la interfaz es el siguiente:

```
#!/usr/bin/env python3
"""Monitor en tiempo real para el sistema de control de la practica
   02."""

import argparse
import csv
import queue
import re
import threading
import time
import tkinter as tk
from collections import deque
from tkinter import filedialog, messagebox, ttk

import serial
from serial.tools import list_ports


DATA_PATTERN = re.compile(
    r"DATA\s*,\s*([-+]?\d+(?:\.\d+)?)\s*,\s*"
    r"([-+]?\d+(?:\.\d+)?)\s*,\s*([-+]?\d+(?:\.\d+)?)"
)
LEGACY_PATTERN = re.compile(
    r"x\[k\]\s*:\s*([-+]?\d+(?:\.\d+)?)\s*\|\s*"
    r"y\[k\]\s*:\s*([-+]?\d+(?:\.\d+)?)"
)


class Monitor:
    def __init__(self, root, initial_port=None):
```

```
30          self.root = root
31          self.root.title("Practica 02 | Monitor de control")
32          self.root.geometry("980x700")
33          self.root.minsize(760, 520)
34
35          self.serial_port = None
36          self.reader_thread = None
37          self.stop_reader = threading.Event()
38          self.samples = deque(maxlen=1200)
39          self.incoming = queue.Queue()
40          self.paused = False
41          self.fallback_time = 0.0
42          self.first_device_time = None
43
44          self.port_var = tk.StringVar(value=initial_port or self.
               detect_port())
45          self.status_var = tk.StringVar(value="Desconectado")
46          self.count_var = tk.StringVar(value="Muestras: 0")
47
48          self.build_controls()
49          self.build_plot()
50          self.root.protocol("WM_DELETE_WINDOW", self.close)
51          self.refresh_plot()
52          if self.port_var.get():
53              self.root.after(150, self.auto_connect)
54
55      @staticmethod
56      def detect_port():
57          ports = [port.device for port in list_ports.comports()]
58          return ports[0] if ports else "/dev/ttyACM0"
59
60      def build_controls(self):
61          controls = ttk.Frame(self.root, padding=(10, 8))
62          controls.pack(fill=tk.X)
63
64          ttk.Label(controls, text="Puerto:").pack(side=tk.LEFT)
65          self.port_combo = ttk.Combobox(
66              controls, textvariable=self.port_var, width=18
67          )
68          self.port_combo.pack(side=tk.LEFT, padx=(5, 8))
69          ttk.Button(controls, text="Actualizar", command=self.
               refresh_ports).pack(
70              side=tk.LEFT, padx=(0, 8)
71          )
72          self.connect_button = ttk.Button(
73              controls, text="Conectar", command=self.
                   toggle_connection
74          )
```

```
75          self.connect_button.pack(side=tk.LEFT)
76          ttk.Button(controls, text="Pausar", command=self.
               toggle_pause).pack(
77              side=tk.LEFT, padx=5
78          )
79          ttk.Button(controls, text="Limpiar", command=self.
               clear_samples).pack(
80              side=tk.LEFT
81          )
82          ttk.Button(controls, text="Guardar CSV", command=self.
               save_csv).pack(
83              side=tk.LEFT, padx=5
84          )
85
86          ttk.Label(controls, textvariable=self.count_var).pack(side=
               tk.RIGHT)
87          ttk.Label(self.root, textvariable=self.status_var, padding
               =(10, 0)).pack(
88              anchor=tk.W
89          )
90
91      def build_plot(self):
92          self.canvas = tk.Canvas(
93              self.root, background="#ffffff", highlightthickness=1,
94              highlightbackground="#c8c8c8"
95          )
96          self.canvas.pack(fill=tk.BOTH, expand=True, padx=10, pady=8)
97          self.canvas.bind("<Configure>", lambda event: self.
               update_lines())
98
99      def refresh_ports(self):
100         ports = [port.device for port in list_ports.comports()]
101         self.port_combo["values"] = ports
102         if ports and self.port_var.get() not in ports:
103             self.port_var.set(ports[0])
104         self.status_var.set(
105             "Puertos actualizados" if ports else "No se encontraron
                   puertos seriales"
106         )
107
108     def toggle_connection(self):
109         if self.serial_port is None:
110             self.connect()
111         else:
112             self.disconnect()
113
114     def auto_connect(self):
115         if self.serial_port is not None:
```

```
116             return
117         if self.connect(show_errors=False):
118             return
119         self.status_var.set("Esperando el puerto serial...")
120         self.root.after(500, self.auto_connect)
121 
122     def connect(self, show_errors=True):
123         port = self.port_var.get().strip()
124         if not port:
125             if show_errors:
126                 messagebox.showerror("Puerto serial", "Selecciona un
                        puerto de la Pico.")
127             return False
128         try:
129             self.serial_port = serial.Serial(port, 115200, timeout
                    =0.2)
130         except serial.SerialException as error:
131             self.serial_port = None
132             if show_errors:
133                 messagebox.showerror("No se pudo conectar", str(
                        error))
134             return False
135 
136         self.stop_reader.clear()
137         self.reader_thread = threading.Thread(target=self.
                read_serial, daemon=True)
138         self.reader_thread.start()
139         self.connect_button.configure(text="Desconectar")
140         self.status_var.set(f"Conectado a {port} | esperando
                muestras")
141         return True
142 
143     def disconnect(self):
144         self.stop_reader.set()
145         if self.serial_port is not None:
146             self.serial_port.close()
147         self.serial_port = None
148         self.connect_button.configure(text="Conectar")
149         self.status_var.set("Desconectado")
150 
151     def read_serial(self):
152         while not self.stop_reader.is_set() and self.serial_port is
                not None:
153             try:
154                 line = self.serial_port.readline().decode("utf-8",
                        errors="ignore")
155             except (serial.SerialException, OSError):
156                 break
```

```
157             if line:
158                 sample = self.parse_sample(line)
159                 if sample is not None:
160                     self.incoming.put(sample)
161 
162     def parse_sample(self, line):
163         match = DATA_PATTERN.search(line)
164         if match:
165             device_time, input_value, output_value = map(float,
                    match.groups())
166             if self.first_device_time is None:
167                 self.first_device_time = device_time
168             elapsed = (device_time - self.first_device_time) /
                    1000.0
169             return elapsed, input_value, output_value
170 
171         # Permite usar versiones anteriores del firmware que
                imprimen x[k] e y[k].
172         match = LEGACY_PATTERN.search(line)
173         if match:
174             input_value, output_value = map(float, match.groups())
175             self.fallback_time += 0.025
176             return self.fallback_time, input_value, output_value
177         return None
178 
179     def toggle_pause(self):
180         self.paused = not self.paused
181         self.status_var.set("Gráfica pausada" if self.paused else "
                Gráfica activa")
182 
183     def clear_samples(self):
184         self.samples.clear()
185         self.first_device_time = None
186         self.fallback_time = 0.0
187         self.update_lines()
188         self.count_var.set("Muestras: 0")
189 
190     def refresh_plot(self):
191         while True:
192             try:
193                 sample = self.incoming.get_nowait()
194             except queue.Empty:
195                 break
196             if not self.paused:
197                 self.samples.append(sample)
198 
199         if not self.paused:
200             self.update_lines()
```

```
201             self.root.after(50, self.refresh_plot)
202
203         def update_lines(self):
204             self.canvas.delete("all")
205             width = max(self.canvas.winfo_width(), 500)
206             height = max(self.canvas.winfo_height(), 400)
207             samples = list(self.samples)
208             self.draw_panel(
209                 0, 0, width, height // 2, samples, 1, "Entrada x[k]", "
                        #1769aa"
210             )
211             self.draw_panel(
212                 0, height // 2, width, height, samples, 2, "Salida y[k]"
                        , "#d1495b"
213             )
214             self.count_var.set(f"Muestras: {len(self.samples)}")
215
216         def draw_panel(self, left, top, right, bottom, samples,
                value_index, title, color):
217             margin_left, margin_top = 72, 28
218             margin_right, margin_bottom = 20, 30
219             plot_left = left + margin_left
220             plot_top = top + margin_top
221             plot_right = right - margin_right
222             plot_bottom = bottom - margin_bottom
223             self.canvas.create_text(
224                 plot_left, top + 10, text=title, anchor=tk.W,
225                 font=("TkDefaultFont", 10, "bold")
226             )
227             self.canvas.create_rectangle(plot_left, plot_top, plot_right
                    , plot_bottom, outline="#aaaaaa")
228             for fraction in (0.0, 0.5, 1.0):
229                 y = plot_bottom - fraction * (plot_bottom - plot_top)
230                 self.canvas.create_line(plot_left, y, plot_right, y,
                        fill="#e5e5e5")
231
232             if not samples or plot_right <= plot_left or plot_bottom <=
                    plot_top:
233                 return
234             times = [sample[0] for sample in samples]
235             values = [sample[value_index] for sample in samples]
236             right_time = max(30.0, times[-1])
237             left_time = max(0.0, right_time - 30.0)
238             maximum = max(1.0, max(values) * 1.1)
239
240             points = []
241             time_span = max(1.0, right_time - left_time)
242             height_span = plot_bottom - plot_top
```

```
243             for sample_time, value in zip(times, values):
244                 x = plot_left + (sample_time - left_time) / time_span *
                        (plot_right - plot_left)
245                 y = plot_bottom - (value / maximum) * height_span
246                 points.extend((x, y))
247             if len(points) >= 4:
248                 self.canvas.create_line(*points, fill=color, width=2,
                        smooth=False)
249             self.canvas.create_text(plot_left - 8, plot_top, text=f"{
                    maximum:.0f}", anchor=tk.E)
250             self.canvas.create_text(plot_left - 8, plot_bottom, text="0"
                    , anchor=tk.E)
251             self.canvas.create_text(plot_right, bottom - 8, text=f"{
                    right_time:.1f} s", anchor=tk.E)
252 
253     def save_csv(self):
254         if not self.samples:
255             messagebox.showinfo("Guardar CSV", "Todavía no hay
                        muestras para guardar.")
256             return
257         filename = filedialog.asksaveasfilename(
258             title="Guardar muestras",
259             defaultextension=".csv",
260             filetypes=[("CSV", "*.csv"), ("Todos los archivos", "*.*
                        ")],
261         )
262         if not filename:
263             return
264         with open(filename, "w", newline="", encoding="utf-8") as
                output:
265             writer = csv.writer(output)
266             writer.writerow(("tiempo_s", "entrada_x_k", "salida_y_k"
                        ))
267             writer.writerows(self.samples)
268         self.status_var.set(f"Datos guardados en {filename}")
269 
270     def close(self):
271         self.disconnect()
272         self.root.destroy()
273 
274 
275 def main():
276     parser = argparse.ArgumentParser(description=__doc__)
277     parser.add_argument("port", nargs="?", help="Puerto serial, por
            ejemplo /dev/ttyACM0")
278     args = parser.parse_args()
279     root = tk.Tk()
280     Monitor(root, args.port)
```

```
    root.mainloop()


if __name__ == "__main__":
    main()
```

Listing 3: Interfaz Python para graficar la entrada y la salida

8 Resultados

Al iniciar el programa se muestra el mensaje de arranque y después se envía una línea serial por cada muestra:

```
Sistema de control de segundo orden iniciado a 40 Hz...
DATA,tiempo_ms,entrada,salida
```

La lectura se actualiza nominalmente cada 25 ms. El filtro EMA reduce las variaciones rápidas del potenciómetro y los polos de magnitud 0.8 evitan el crecimiento de la respuesta. La señal `x[k]` corresponde a la entrada filtrada del ADC, mientras que `y[k]` es la salida calculada mediante la ecuación de diferencias; no representa una tensión medida en un pin de salida.

8.1 Pruebas con diferentes coeficientes

Para comparar la respuesta del sistema se realizaron tres pruebas. En cada caso se modificaron los coeficientes del programa, se volvió a cargar el firmware en la Pico y se guardó una captura de la interfaz. Las imágenes deben colocarse en la carpeta `practica_02/img/` con los nombres indicados.

Prueba	b_0	b_1	b_2	a_1	a_2	Comportamiento
1	0.010	0.020	0.010	-1.6	0.64	Respuesta base
2	0.040	0.080	0.040	-1.2	0.36	Más rápida y amortiguada
3	0.005	0.010	0.005	-1.8	0.81	Más lenta

Table 1: Coeficientes utilizados en las pruebas experimentales.

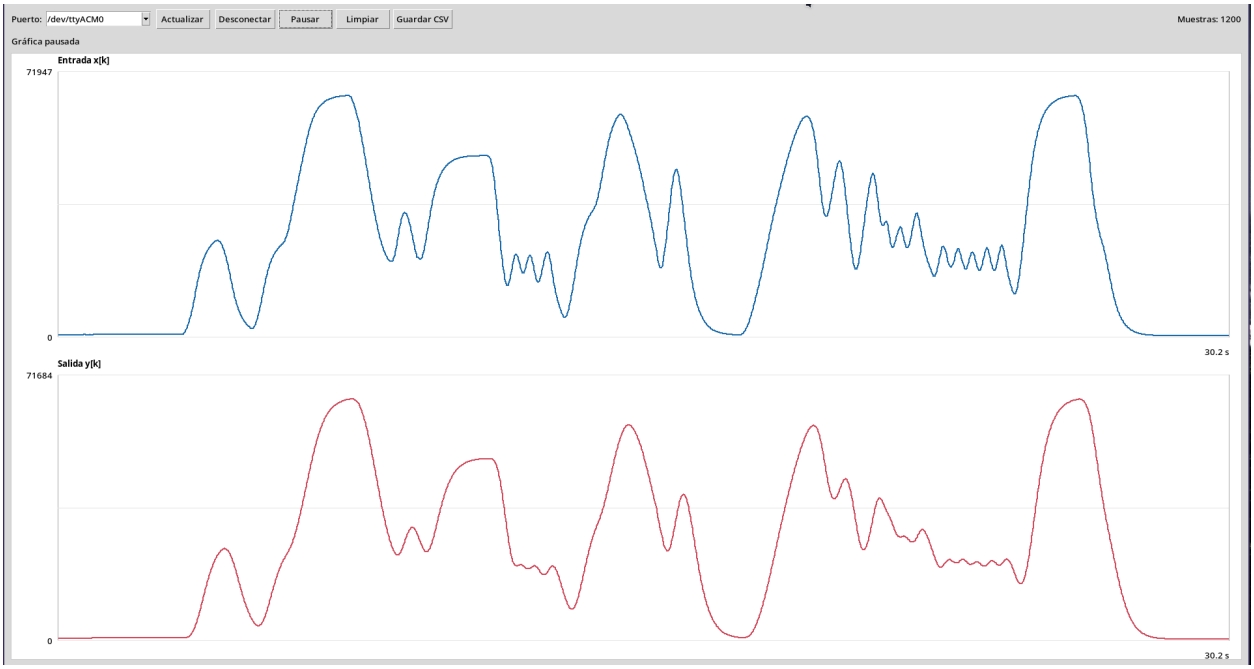


Figure 1: Respuesta del sistema con los coeficientes de la prueba 1.

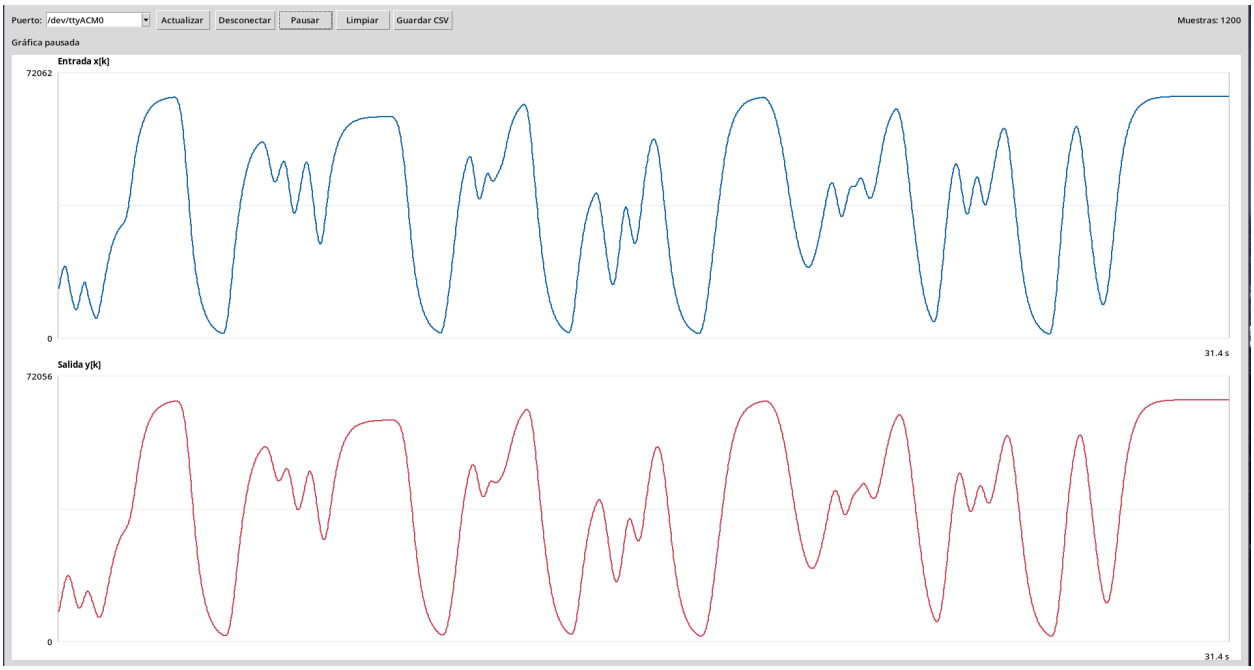


Figure 2: Respuesta del sistema con los coeficientes de la prueba 2.

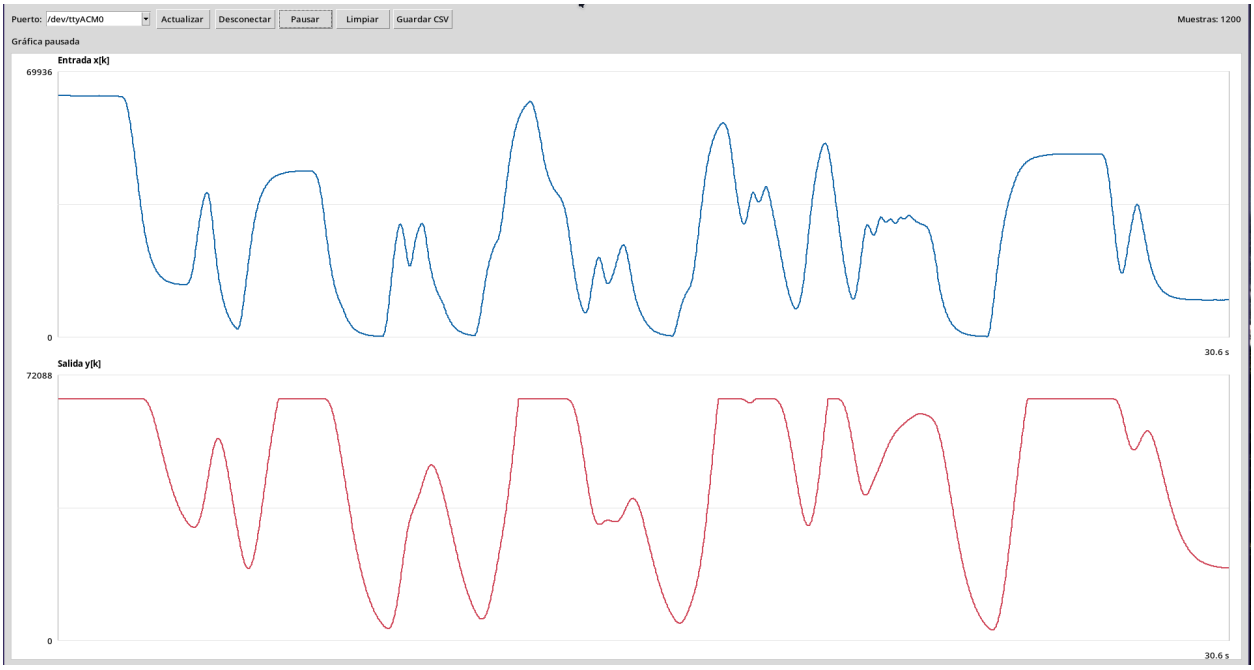


Figure 3: Respuesta del sistema con los coeficientes de la prueba 3.

Las tres pruebas confirmaron que la posición de los polos modifica directamente la rapidez de respuesta del sistema. Los polos de magnitud 0.6 produjeron la respuesta más rápida, los polos de magnitud 0.8 produjeron una respuesta intermedia y los polos de magnitud 0.9 produjeron la respuesta más lenta. En los tres casos el sistema permaneció estable porque los polos se mantuvieron dentro del círculo unitario.

La salida también puede almacenarse en un archivo desde la terminal:

```
../upload.sh source/practica_02.py >> source/output.txt
```

9 Conclusiones individuales

9.1 Ayala Hernández María Fernanda

La práctica permitió implementar un sistema de segundo orden en tiempo discreto sobre una Raspberry Pi Pico 2 W. El uso del ADC y del temporizador mostró cómo un microcontrolador puede adquirir señales y procesarlas con un periodo de muestreo definido.

9.2 Salazar Islas Luis Daniel

La implementación permitió relacionar una función de transferencia con una ecuación de diferencias ejecutable en MicroPython. También se observó que la elección de los polos es

fundamental para evitar que la respuesta digital sea inestable.

9.3 Tepal Briseño Hansel Yael

El filtrado exponencial ayudó a reducir el ruido producido por el potenciómetro. Esta práctica mostró que la estabilidad depende tanto del algoritmo de control como de la calidad de la señal de entrada y de la conexión física del circuito.

9.4 Ugartechea González Luis Antonio

La ejecución en la Raspberry Pi Pico 2 W permitió comprobar la interacción entre el software y el hardware. El temporizador periódico garantiza el ritmo de procesamiento, mientras que la salida en la terminal facilita observar el comportamiento del sistema durante las pruebas.